

VVS Assignment 2: Comprehensive Testing of the Calendar Application

Rodrigo Neto

June 6, 2026

1 Introduction

This report describes the testing strategy and implementation for the *Calendar* application. The goal was to provide a robust test suite covering multiple levels of the application: Unit Testing, Integration Testing (3rd party, REST, and Database), and End-to-End (E2E) testing, along with Continuous Integration setup.

2 Step-by-Step Solution

The following steps were taken to fulfill the assignment requirements:

1. **Environment Setup:** Added Maven dependencies for JUnit 5, Mockito, Selenium, WebDriverManager, and Spring Security Test.
2. **SUT Adaptation:** Modified the source code to improve testability, specifically injecting `RestClient.Builder` in discovery providers and adding `setId` methods to model classes.
3. **Unit Testing:** Developed tests for services using Mockito to isolate business logic from persistence.
4. **3rd Party Integration:** Used `@RestClientTest` to mock external API responses and verify parsing logic.
5. **REST API Integration:** Used `MockMvc` and `@WebMvcTest` to test controller endpoints and security.
6. **Database Integration:** Verified the JPA layer using `@SpringBootTest` with an H2 in-memory database.
7. **End-to-End Testing:** Created a full flow automation using Selenium to verify the user journey from registration to meeting creation.
8. **CI Setup:** Configured GitHub Actions to automate test execution.

3 Testing Details

3.1 Unit Testing (Business Logic)

Targeting the `service` package.

- **UserServiceTest:** Tests user registration (success and duplicate username cases) and user retrieval.

- **MeetingServiceTest**: Tests proposing meetings (valid/invalid times, unknown users), responding to invites, and copying events from discovery.
- **AppUserDetailsServiceTest**: Ensures Spring Security correctly loads user details from the repository.
- **ICalServiceTest**: Verifies that the generated iCal feed contains expected VCALENDAR headers and event details.

3.2 Integration Testing (3rd Party Sources)

Targeting the `discover` package using `MockRestServiceServer`.

- **AgendaLxProviderIntegrationTest**: Verifies parsing of Lisbon's Agenda Cultural JSON.
- **TicketmasterProviderIntegrationTest**: Verifies parsing of Ticketmaster API responses.
- **SeatGeekProviderIntegrationTest**: Verifies parsing of SeatGeek API responses.

3.3 REST API Integration (Controllers)

Targeting the `controller` package using `MockMvc`.

- **AuthControllerTest**: Tests `/login` and `/register` routes, including failure scenarios (e.g., username taken).
- **MeetingControllerTest**: Tests the meeting proposal flow, ensuring redirection to calendar on success and re-rendering the form with errors on failure.

3.4 Database Integration

- **MeetingServiceIntegrationTest**: Uses `@SpringBootTest` to perform a complete cycle of proposing and responding to a meeting, asserting that the state changes correctly in the underlying H2 database.

3.5 End-to-End Testing (Selenium)

- **FullFlowE2ETest**: Uses `HtmlUnitDriver` to perform a full system test:
 1. Navigate to `/register` and create a new unique user.
 2. Log in with the new credentials.
 3. Navigate to `/meetings/new` and propose a meeting using JavaScript to handle `datetime-local` inputs.
 4. Verify that the meeting title appears on the user's calendar.

4 Modifications to the SUT

To enable effective testing, the following changes were made:

- **Discovery Providers**: Changed from manual `RestClient` instantiation to `RestClient.Builder` constructor injection. This allows Spring's `@RestClientTest` to intercept and mock HTTP calls.
- **Model Classes**: Added `setId(Long id)` to `User` and `Meeting`. This was necessary to mock persistence-assigned IDs in unit and REST tests.

5 How to Run the Tests

Ensure you have Maven and a JDK (17+) installed.

1. Navigate to the `calendar` directory.
2. Run all tests: `mvn test`
3. To run a specific test class: `mvn test -Dtest=ClassName`

6 Conclusion

The resulting test suite provides high coverage across all application layers. The combination of fast unit tests and comprehensive E2E tests ensures that both business logic and user-facing features are reliable.